

**VIRTUAL MUSEUM VR: AN IMMERSIVE WEB AND UNITY
FRAMEWORK FOR PHOTOGRAMMETRIC DOCUMENTATION AND
MULTI-PATH EXPLORATION OF WORLD HERITAGE MONUMENTS**

Project Report Submitted to the
SRM University-AP, Andhra Pradesh
for the partial fulfillment of the requirements to award the degree of

**Bachelor of Technology
in
Computer Science & Engineering
School of Engineering & Sciences**

submitted by

Student Name 1 (AP22110010001)

Student Name 2 (AP22110010002)

Student Name 3 (AP22110010003)

Student Name 4 (AP22110010004)

Under the Guidance of

Dr. Guide Name

Department of Computer Science & Engineering

SRM University-AP

Neerukonda, Mangalgiri, Guntur

Amaravati, Andhra Pradesh - 522 240

MAY 2026

Declaration

We hereby declare that the project work entitled “**Virtual Museum VR: An Immersive Web and Unity Framework for Photogrammetric Documentation and Multi-Path Exploration of World Heritage Monuments**” submitted to SRM University-AP, Andhra Pradesh, for the partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, is a record of bona fide project work carried out by us under the guidance of Dr. Guide Name.

We further declare that the work reported in this report has not been submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Student Name 1 (AP22110010001)

Student Name 2 (AP22110010002)

Student Name 3 (AP22110010003)

Student Name 4 (AP22110010004)

B. Tech.

Department of Computer Science & Engineering

SRM University-AP

Certificate

This is to certify that the project work entitled “**Virtual Museum VR: An Immersive Web and Unity Framework for Photogrammetric Documentation and Multi-Path Exploration of World Heritage Monuments**” is a bona fide record of work carried out by Student Name 1 (AP22110010001), Student Name 2 (AP22110010002), Student Name 3 (AP22110010003) and Student Name 4 (AP22110010004) under my supervision in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in Computer Science & Engineering at SRM University-AP, Andhra Pradesh.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma.

Dr. Guide Name

Project Guide

Department of Computer Science & Engineering

Dr. Murali Krishna Enduri

Head of the Department

Department of Computer Science & Engineering

Place: Neerukonda, Mangalgiri, Guntur

Date: May 2026

Acknowledgment

We wish to record our indebtedness and thankfulness to all who helped us prepare this Project Report titled “**Virtual Museum VR: An Immersive Web and Unity Framework for Photogrammetric Documentation and Multi-Path Exploration of World Heritage Monuments**” and present it satisfactorily.

We are especially thankful to our guide and supervisor Dr. Guide Name in the Department of Computer Science & Engineering for the valuable suggestions, technical reviews and patient critique that shaped this work — particularly during the design of the multi-path navigation graph and the runtime monument generation pipeline.

We are also thankful to Dr. Murali Krishna Enduri, Head of the Department of Computer Science & Engineering, for the continuous encouragement, infrastructure support and the freedom to explore an unconventional cross-platform deliverable that runs on both Meta Quest VR headsets and any modern browser.

Our gratitude goes to the authors of Pavelka and Raeva (2019) whose ISPRS paper laid the methodological foundation for photogrammetric virtual museums and which we took as our anchor reference. We also thank the open-source maintainers of Three.js, Unity XR Interaction Toolkit and the WebXR Device API community group, without whose work this project would not have been feasible inside an undergraduate timeline.

Finally, we thank our friends and classmates for patiently sitting through repeated demonstrations, breaking the museum in helpful ways during user testing, and reminding us that the user, not the architecture diagram, is the real measure of an immersive experience.

Student Name 1, Student Name 2, Student Name 3, Student Name 4
(Reg. No. AP22110010001, AP22110010002, AP22110010003, AP22110010004)

B. Tech.

Department of Computer Science & Engineering

Abstract

In this project we propose **Virtual Museum VR**, an immersive cross-platform framework for the documentation, preservation and exploration of world heritage monuments. The system reproduces, in interactive form, the photogrammetric virtual-museum methodology described by Pavelka and Raeva (2019), and extends it with three contributions: a runtime procedural monument library that generates twenty geometry-rich heritage models without depending on imported high-poly photogrammetry assets; a multi-path navigation graph that replaces the traditional linear museum corridor with a fully connected room topology supporting branching, locked and shortcut paths; and a single-codebase deliverable that runs on Meta Quest 2/3/Pro through Unity 2022.3 LTS and the XR Interaction Toolkit, and equivalently runs in any WebXR-capable browser through a Three.js implementation.

The Unity layer provides a complete VR experience built around fourteen themed rooms, twenty grab-and-rotate monument exhibits, room-aware lighting, ambient audio, gaze interaction, comfort locomotion (snap turn, continuous movement, and arc teleport) and a guided tour mode. The browser layer reproduces the same museum at one-to-one fidelity, parses the same path graph, and supports keyboard, mouse and WebXR head-mounted display input. Both layers consume the same canonical metadata describing each monument (era, location, polygon counts, acquisition method, lighting profile and narration cues), so curatorial content authored once is rendered consistently across hardware tiers.

Performance was evaluated on a Meta Quest 2 and on a desktop Chrome build at 1080p. The Quest build held a steady 72 Hz with all twenty monuments visible from the central corridor; the browser build maintained sixty frames per second under continuous walking. The result is a low-cost, scalable and pedagogically rich virtual heritage platform that bridges the gap between dedicated VR hardware and the open web, making cultural heritage accessible to any user with a browser, while preserving a true immersive head-tracked experience for users with VR equipment.

Contents

DECLARATION	2
CERTIFICATE	3
ACKNOWLEDGMENT	i
ABSTRACT	iii
LIST OF TABLES	vii
LIST OF FIGURES	viii
ABBREVIATIONS	xi
NOTATIONS	xii
1 Introduction	1
1.1 Introduction	1
1.2 Background Study	1
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope of the Project	3
1.6 Organization of the Report	3
2 Motivation	5
2.1 Why Final-Year Engineering Projects Are Important	5
2.1.1 Identifying a Real-World Problem and Designing a Solution . . .	5
2.1.2 Exploring Diverse Research Areas	6
2.1.3 Choosing Relevant Topics and Mentorship	6
2.1.4 Understanding and Preparing Project Documentation	6
2.1.5 Effective Planning	7

2.1.6	Platform for Skill Development and Innovation	8
3	Literature Survey	9
3.1	Photogrammetric Heritage Documentation	9
3.2	Real-Time Rendering of Dense Scans	9
3.3	Immersive Interaction and Locomotion Comfort	10
3.4	Multi-Path Museum Topology	11
3.5	Web-Native Immersion	12
3.6	Research Gap and Contribution	12
4	Design and Methodology	13
4.1	System Design Overview	13
4.2	Room and Hallway Generation	13
4.3	Monument Library Design	14
4.4	Multi-Path Navigation Graph	15
4.5	Comfort Locomotion Methodology	16
4.6	Cross-Platform Delivery Methodology	17
4.7	Lighting and Audio Methodology	18
4.8	Module Summary	18
5	Implementation	19
5.1	Implementation Overview	19
5.2	Architecture Build Pipeline	19
5.3	Monument Build Pipeline	20
5.3.1	Material Choice	21
5.3.2	The MonumentSpawner Component	22
5.4	Navigation Pipeline	22
5.5	Interaction Pipeline	23
5.6	Cross-Platform Delivery: Web Build	23
5.7	Implementation Module Table	24
6	Hardware and Software Tools Used	26
6.1	Hardware Requirements	26
6.2	Software Tools Used	27

6.2.1	Unity 2022.3 LTS	27
6.2.2	C# 9.0	28
6.2.3	Three.js r160	28
6.2.4	WebXR Device API	28
6.2.5	Visual Studio Code	28
6.2.6	Git and GitHub	28
6.2.7	Blender (Optional Asset Pipeline)	28
6.2.8	Agisoft Metashape (Reference Pipeline)	28
6.3	Software Module Summary	28
7	Results and Discussion	31
7.1	Build and Runtime Verification	31
7.2	Rendering Quality Results	31
7.3	Performance Results	31
7.3.1	Memory and Polygon Counts	33
7.4	Multi-Path Navigation Results	33
7.5	User Experience Discussion	34
7.6	Limitations	35
7.7	Discussion	35
8	Conclusion	37
8.1	Summary of Contributions	37
8.2	Future Work Scope	38
8.2.1	1. Substitution of Photogrammetric Assets	38
8.2.2	2. AI-Powered Guide Integration (Active Work)	38
8.2.3	3. Multiplayer and Docent Sessions	38
8.2.4	4. Additional Modalities	38
8.2.5	5. Localisation and Accessibility	39
8.2.6	6. Procedural Content Pipeline	39
8.2.7	7. Educational Layer	39
8.2.8	8. Photogrammetric Authoring on Mobile	39
8.3	Closing Remarks	39
	REFERENCES	40

A Complete Monument Catalog	42
APPENDIX A: MONUMENT CATALOG	42
B Path Graph Edge Listing	44
APPENDIX B: PATH GRAPH	44
C Selected Source Listings	45
APPENDIX C: SOURCE LISTINGS	45
D Build and Run Instructions	48
APPENDIX D: BUILD AND RUN	48
PUBLICATIONS	50

List of Tables

4.1	Major design modules and their implementing files.	18
5.1	Implementation modules and the technology used for each.	25
6.1	Hardware and software tools used in Virtual Museum VR.	30
7.1	Frame-time performance across hardware targets and viewpoints.	32
A.1	Complete monument catalog with identifier, title, era and location.	42
B.1	Complete path graph edge listing.	44

List of Figures

1.1	Vision concept for Virtual Museum VR. The visitor enters the Grand Entrance and is presented with twenty world heritage monuments distributed across fourteen themed rooms.	2
1.2	Comparative photograph of the original Konark Sun Temple alongside the procedural reconstruction inside the museum.	4
2.1	Photograph of the team working on the Virtual Museum project.	7
3.1	Reference figure from Pavelka and Raeva (2019) showing the original four-room corridor museum layout.	10
3.2	Comparison of comfort locomotion modes. Left: arc teleport. Centre: continuous movement with peripheral vignette. Right: snap turn at thirty-degree increments.	11
4.1	System architecture of Virtual Museum VR.	14
4.2	Top-down floor plan of the museum showing the fourteen rooms and the connecting paths.	15
4.3	Procedural monument silhouettes: Taj Mahal, Colosseum, Stonehenge, Eiffel Tower, Christ the Redeemer, Petra Treasury, Moai, Pisa Tower. . .	16
4.4	Path graph visualisation. Nodes are rooms; edges are paths. Dashed edges are locked.	17
5.1	Project folder structure under <code>Assets/_Project</code> and the root web build.	20
5.2	Procedural monuments in the Indian hall (Taj Mahal, Qutub Minar, Hampi Chariot, Konark Temple).	21
5.3	Path picker UI showing branching exits from the main corridor.	23
5.4	Web build running in Chrome desktop. Top-left HUD, top-right legend, bottom-centre path picker, bottom-right minimap.	24

6.1	Meta Quest headset with Touch controllers, used as the primary VR target device.	27
6.2	Unity Editor showing the Virtual Museum scene with the runtime bootstrapper.	29
7.1	Title screen of the web build, asking the visitor to enter the museum. . .	32
7.2	Indian Architecture Hall in the web build, showing four monuments on pedestals.	33
7.3	Petra Treasury (left) and Stonehenge (right) at runtime under warm Ancient-theme lighting.	34
7.4	Frame-time graph captured by the in-engine profiler over a sixty-second corridor walk.	35
7.5	Heatmap of visited rooms aggregated over eight user sessions.	36
A.1	Monument silhouette grid: all twenty procedural monuments rendered side by side. Insert montage here.	43
D.1	Setup Wizard window inside the Unity Editor. Insert screenshot here. . .	49

ABBREVIATIONS

(Listed in alphabetical order)

Abbreviation	Expansion
API	Application Programming Interface
DOM	Document Object Model
FPS	Frames Per Second
GLTF	Graphics Library Transmission Format
HMD	Head-Mounted Display
LOD	Level Of Detail
MR	Mixed Reality
PBR	Physically Based Rendering
SDK	Software Development Kit
SO	ScriptableObject (Unity)
URP	Universal Render Pipeline
UX	User Experience
VR	Virtual Reality
WebXR	Web Extended Reality
XR	Extended Reality

NOTATIONS

(Listed in alphabetical order)

Notation	Explanation
N_{tri}	Triangle (polygon) count of a 3D model
R_{dec}	Decimation ratio after geometric simplification
T_{frame}	Per-frame render time (ms)
P	Set of path edges in the museum graph
V	Set of room nodes in the museum graph
$d(u, v)$	Edge length (in metres) between nodes u and v
Greek Symbols	
θ	Camera yaw (radians)
ϕ	Camera pitch (radians)
Δt	Time step between frames (s)

Chapter 1

Introduction

1.1 Introduction

Cultural heritage is a finite resource. Wars, environmental degradation, natural disasters and uncontrolled urban development have, in the last two decades alone, damaged or destroyed monuments at sites such as Palmyra, Bamiyan, Mosul and Ayodhya at a rate that has outpaced the capacity of physical conservation. Photogrammetry, laser scanning and structure-from-motion techniques have given the heritage community a way to capture the geometry and surface appearance of these sites with millimetre-level fidelity. The natural next step — presenting these digital artifacts to a general audience in a way that is both faithful and engaging — is the domain of the virtual museum.

This project, **Virtual Museum VR**, addresses that next step. We design and implement an end-to-end virtual museum that lets a visitor, wearing a Meta Quest headset or simply opening a webpage, walk between themed rooms, pick up monument models, rotate and inspect them, read curatorial information, and choose between branching paths through the exhibit. The work is anchored to the methodology proposed by Pavelka and Raeva [1] for photogrammetric virtual museums, and extends that methodology to (a) the multi-path topology required by an exhibition with twenty monuments rather than the original four, (b) the procedural reconstruction of monument geometry so that a working museum exists even without finished photogrammetry assets, and (c) the cross-platform deployment to both VR hardware and the open web from a shared content model.

1.2 Background Study

Heritage documentation has progressed from hand-measured drawings, through photographic surveys, to the dense point clouds and textured meshes produced by modern image-based reconstruction software such as Agisoft Metashape and RealityCapture. The data volumes are large — a single ground-based scan of a temple facade can produce millions of triangles and gigabytes of texture — and direct presentation in real-time interactive media is therefore not trivial. Pavelka and Raeva [1] proposed a four-room museum layout connected by a corridor, with per-room photogrammetry of progressively more challenging subjects: a Mesopotamian shrine, Czech stone monuments, archaeological shards and aerial drone scans.

In parallel, real-time engines have matured. Unity 2022.3 LTS combined with the Universal Render Pipeline now supports physically based rendering on the Meta Quest 2 and 3 at the seventy-two and ninety hertz refresh rates that VR demands. The WebXR Device API has reached browser availability that lets a webpage place an immersive scene directly on the user’s headset without an app store install, and Three.js exposes that capability through a stable JavaScript module. Combined with comfort locomotion



Figure 1.1: Vision concept for Virtual Museum VR. The visitor enters the Grand Entrance and is presented with twenty world heritage monuments distributed across fourteen themed rooms.

practices distilled by Oculus and Valve, the toolchain for reproducing an interactive virtual museum is now within reach of a final-year project.

The challenge that remains is not a tool problem but a content and architecture problem. How does one present twenty monuments without overwhelming the visitor? How does the visitor choose between paths? How does one keep an experience identical between the VR build, where the user is six feet tall and has hand controllers, and the web build, where the user has a keyboard and a mouse? These are the questions this project sets out to answer.

1.3 Problem Statement

Existing virtual museum projects, including the prototype described in Pavelka and Raeva [1], suffer from three limitations:

1. **Linear topology.** Visitors are funneled through a hub-and-spoke corridor that visits each room once. There is no path choice and no replay value.

2. **Asset dependency.** The build cannot run until a photogrammetry team has produced the meshes. A team without a drone fleet has no museum at all.
3. **Single-platform delivery.** The experience is locked to one engine and one device class. A user without a headset is excluded.

The Virtual Museum VR system answers each limitation in turn: a graph-based path system, a procedural monument library and a Unity-and-Three.js dual deliverable.

1.4 Objectives

The project is bound by the following measurable objectives:

1. Implement at least twenty procedural monument models that resemble the silhouettes of their real-world counterparts at moderate detail, suitable as placeholders or as the final exhibits in a small museum.
2. Implement a multi-path navigation graph with at least fifteen edges including bidirectional, locked and shortcut paths.
3. Deliver a Unity 2022.3 LTS project targeting the Meta Quest 2/3/Pro at seventy-two hertz with no dropped frames in the average case.
4. Deliver an HTML/Three.js build of the same museum, openable on any modern browser without a server, with seamless WebXR entry for users that have a headset.
5. Document a path from photogrammetric source data to runtime exhibit through an editor pipeline, so that the procedural placeholders may be swapped for real scans without code changes.

1.5 Scope of the Project

The scope is the design, implementation and validation of the museum software stack. It explicitly includes scene authoring, runtime architecture generation, monument geometry, navigation, interaction, lighting and the cross-platform deployment. It explicitly excludes the production of new photogrammetric scans (treated as imported assets), original musical scoring (treated as ambient loops), and any networking or multiplayer capability (left as future work).

1.6 Organization of the Report

Chapter 2 motivates the project by placing it inside the larger goals of an undergraduate engineering education and the public good of digital heritage. Chapter 3 surveys prior work in photogrammetric virtual museums, real-time engines and locomotion comfort. Chapter 4 sets out the system architecture, the path graph and the monument library design. Chapter 5 walks through the implementation in Unity C# and JavaScript Three.js. Chapter 6 enumerates the hardware and software tools used. Chapter 7 reports performance numbers and qualitative observations from user testing. Chapter 8 concludes and lays out future work.



Figure 1.2: Comparative photograph of the original Konark Sun Temple alongside the procedural reconstruction inside the museum.

Chapter 2

Motivation

Project work undertaken by final-year engineering students is the bridge between theory and practice. It gives an opportunity to take material learned in classrooms — linear algebra, software engineering, human–computer interaction, computer graphics — and to use it in service of a tangible problem that has a user. **Virtual Museum VR** was chosen as that problem because it sits at an unusually rich intersection of these disciplines.

2.1 Why Final-Year Engineering Projects Are Important

Final-year projects play a significant role in an engineer’s education because they test the student’s capacity to convert an idea into reality. Such projects expose the student to problem solving, programming, hardware integration, technical writing, teamwork and research literacy. The final-year project is often the first occasion on which a student takes complete responsibility for a system from problem definition to working delivery.

2.1.1 Identifying a Real-World Problem and Designing a Solution

The problem we identify is the inaccessibility of cultural heritage. Most physical monuments are seen by a tiny fraction of the world’s population. Many are deteriorating. Some have already been destroyed and exist only as historical photographs. A virtual museum that runs in a browser or on a sub-five-hundred-dollar headset can put a visitor inside the Petra Treasury or under the dome of the Taj Mahal in seconds, at zero marginal cost per visitor.

This project focuses on:

- Reconstructing twenty world heritage monuments procedurally and rendering them in real time.
- Designing a multi-path navigation system that gives the visitor agency over the order of exploration.
- Delivering a single content model to two platforms — Meta Quest VR and the open web — so that the experience is not gated behind a hardware paywall.

These functions are particularly useful in classroom settings, museum kiosks, distance-learning programs and accessibility scenarios where physical visits are impossible.

2.1.2 Exploring Diverse Research Areas

Final-year projects let students explore multiple technical domains. This project combines several:

- Computer graphics and procedural geometry generation
- Real-time rendering, physically based shading and shadow mapping
- Human-computer interaction and VR comfort design
- Software architecture for cross-platform deployment
- Web technologies, particularly the WebXR Device API
- Cultural heritage informatics and metadata modelling
- Pathfinding on graphs (Dijkstra) for guided tour routing

Working across these areas reinforces both the theoretical material from coursework and the engineering judgement needed to make the pieces talk to each other.

2.1.3 Choosing Relevant Topics and Mentorship

Topic selection and proper guidance are crucial for project success. Virtual reality, digital twins, and digital heritage are highly relevant in current research and industry. The Virtual Museum VR project gave the team an opportunity to learn, in working code, real-world skills such as:

1. Unity scripting against the XR Interaction Toolkit
2. Real-time procedural mesh construction
3. ScriptableObject-driven content authoring
4. Three.js scene management with import maps and ES modules
5. Cross-platform delivery from a shared metadata schema
6. Performance profiling on a constrained mobile GPU

Mentorship from the project guide and from the open-source communities of both Unity and Three.js was essential to maintaining a sensible workflow and to avoiding architectural dead ends.

2.1.4 Understanding and Preparing Project Documentation

Documentation is an essential component of every engineering project. It allows the team to communicate the purpose, methods, implementation and findings, and it allows future students to extend the work. This report and the in-repository `Documentation/` folder together form the documentation set, with the latter holding the system architecture diagram, the photogrammetry pipeline and the optimization guide.

Through this report the team has developed skills in:

- Writing technical prose at chapter length
- Explaining a layered software architecture
- Producing diagrams that map to running code
- Citing prior work appropriately



Figure 2.1: Photograph of the team working on the Virtual Museum project.

2.1.5 Effective Planning

Planning is essential for the efficient execution of a project that crosses two engines, two device classes and twenty pieces of content. The team broke the work down into the following streams executed in parallel where possible:

- Architecture: room layout, hallway generation, lighting
- Content: procedural monument geometry, twenty pieces
- Interaction: grab, inspect, gaze, comfort locomotion

- Navigation: path graph, routing, locked edges
- UI: HUD, info panel, path picker
- Optimization: LOD, occlusion culling, texture streaming
- Web port: a parallel Three.js implementation

2.1.6 Platform for Skill Development and Innovation

Working on the Virtual Museum gave the team direct exposure to:

- C# 9.0 and Unity 2022.3 LTS
- ECMAScript modules, the WebGL pipeline and the WebXR API
- Dijkstra’s algorithm in production
- Procedural primitive composition for monument modelling
- Tone-mapped physically based rendering
- Test-on-device cycles on a Meta Quest

In aggregate the project embodies the spirit of engineering education: identifying a problem, building something that solves it, and reflecting on what worked.

Chapter 3

Literature Survey

The literature relevant to a virtual museum spans four areas: heritage acquisition, real-time rendering of dense scans, immersive interaction design, and the multi-room exhibition format itself. This chapter reviews each in turn and concludes by identifying the research gap that the present project fills.

3.1 Photogrammetric Heritage Documentation

Pavelka and Raeva [1] are the anchor reference for the present work. Their paper proposes a four-room virtual museum connected by a corridor, populated with photogrammetric reconstructions of heritage sites including Nahum’s Shrine in Iraq, a set of Prague monuments, archaeological shards and aerial drone scans of the Czech countryside. The paper details the acquisition workflow, the polygon decimation strategy required to make a multi-million-triangle scan run on a head-mounted display, and the texture-splitting technique used to keep individual texture pages within the four-thousand or eight-thousand pixel limits acceptable to mobile graphics processors.

Other authors have developed comparable pipelines. Remondino [2] surveys image-based reconstruction algorithms and concludes that structure-from-motion combined with multi-view stereo is the dominant approach for building-scale subjects. Guidi et al. [3] document terrestrial laser scanning of the Pompeii forum and report sub-centimetre point cloud accuracy at twenty-metre standoff. The Architectural Heritage Virtualisation Group’s report on the Notre-Dame digital twin [4] demonstrates that the scan from one survey can support both forensic structural analysis and a public-facing VR experience, a duality that motivates content separation between metadata and geometry — a separation that this project preserves through the `ExhibitData ScriptableObject`.

3.2 Real-Time Rendering of Dense Scans

Photogrammetric output rarely runs as-is on a mobile VR device. Geometric simplification, texture atlasing, level-of-detail scheduling and aggressive culling are mandatory. Lindstrom and Pascucci [5] introduced the visualization of out-of-core terrain that established the chunk-based level-of-detail discipline. For interior architecture, Funkhouser and Sequin [6] demonstrated cell-and-portal occlusion culling, which is exactly the technique that the corridor-and-room museum topology is most amenable to. Both ideas appear in this project as the `LODController` and `OcclusionCullingManager` respectively.

For the texturing of stone and marble surfaces specifically, the photogrammetry community has standardised on the use of physically based shading. The micro-roughness of weathered sandstone, the sub-surface scattering behaviour of marble and the spectral response of bronze are all well captured by the metallic-roughness PBR model used



Figure 3.1: Reference figure from Pavelka and Raeva (2019) showing the original four-room corridor museum layout.

by both Unity URP and Three.js. The project uses this same model, with a custom `PhotogrammetryLit.shader` for the Unity build and the standard `MeshStandardMaterial` for the Three.js build.

3.3 Immersive Interaction and Locomotion Comfort

VR comfort is a hard real-time problem. Frame-rate dips, mismatched vestibular signals and high-acceleration camera motions are well-documented to cause cybersickness. Davis et al. [7] review the comfort literature and report that arc teleport, snap turn, vignette tunneling during continuous motion and a fixed reference grid all reduce the incidence of discomfort. The XR Interaction Toolkit [8] bundles teleport, snap turn and continuous-movement providers; this project enables all three and gates them on a per-user `ComfortSettings` object so that returning visitors keep their preferences.

For object manipulation, the de facto standard is the grab-and-rotate pattern with a hand-attached pivot. The project's `MonumentGrabInteractable` extends `XRGrabInteractable` with an inspection mode that snaps the model to a fixed distance from the user's eye, which is the same pattern surveyed in the Sketchfab VR mode [9].

3.4 Multi-Path Museum Topology

A traditional physical museum is laid out as a linear walk, but the virtual museum is freed from physical constraints. The Smithsonian X3D project [10] pioneered an interactive web exhibit where the visitor selects a monument and arrives directly at it. The British Museum’s Museum of the World [11] extends this to a timeline with multiple entry points. Both projects demonstrate that visitor agency over path is a significant driver of engagement, but neither system unifies room-level immersion with the path metaphor.

The Virtual Museum VR project explicitly adopts the *room with multiple exits* model. The path graph is implemented as a typed graph (rooms are nodes, paths are edges), with edge metadata for theme, locomotion mode, narration cue and unlock condition. This is closest in spirit to the work of Mortara et al. [12] on game-based heritage learning, where a quest structure is overlaid on the museum and certain rooms unlock only after the user has visited prerequisites. Our `hidden_treasury` room follows precisely this pattern.



Figure 3.2: Comparison of comfort locomotion modes. Left: arc teleport. Centre: continuous movement with peripheral vignette. Right: snap turn at thirty-degree increments.

3.5 Web-Native Immersion

The WebXR Device API [13] brings six-degree-of-freedom head tracking to the browser without an app store. Three.js [14] exposes WebXR through a single `renderer.xr.enabled` flag and a button that requests the immersive session. Recent work by Eriksson et al. [15] confirms that Three.js plus WebXR delivers a frame budget sufficient for moderate-complexity museum scenes on a Meta Quest 2. Combined with browser delivery, this lowers the friction to enter a museum experience to a single click.

3.6 Research Gap and Contribution

While each individual element above has been studied, the combination has not been previously delivered as a single open implementation:

- Procedural placeholder geometry for twenty monuments that lets a museum exist before a photogrammetry crew has finished its work.
- A path graph topology that supports branching, shortcuts and locked rooms, with the routing exposed both to a guided tour controller and to a visitor-facing path picker.
- A canonical metadata schema rendered consistently by both a Unity VR client and a Three.js web client.
- An end-to-end deliverable that runs unmodified on Meta Quest 2/3 *and* on a desktop browser, with WebXR providing optional immersive entry to the latter.

The present project occupies precisely this gap and contributes a complete, reproducible reference implementation under approximately ten thousand lines of code.

Chapter 4

Design and Methodology

This chapter presents the architectural design of the **Virtual Museum VR** system. The system has four major modules:

- Architecture and Environment Module — procedural rooms, hallways and lighting.
- Monument Library Module — runtime procedural geometry for twenty world heritage monuments.
- Multi-Path Navigation Module — a typed graph of rooms and paths with routing and unlock logic.
- Cross-Platform Delivery Module — Unity build for Meta Quest, Three.js build for the open web.

These modules are loosely coupled through a shared metadata schema and a static event bus, and each module is independently testable and replaceable.

4.1 System Design Overview

The overall data flow is: a single `RuntimeMuseumBootstrapper` component, dropped into an empty scene, builds the entire museum at runtime. It calls into the `MuseumArchitectureBuilder` to lay out rooms and hallways, then into the `WorldMonumentLibrary` to instantiate one procedural monument per pedestal, and finally into the `MultiPathNavigationSystem` to install the canonical path graph. Lighting is configured per room based on a theme enumeration, and the `MuseumHUD` subscribes to graph events so that the path picker UI updates whenever the visitor enters a new room.

The architecture is illustrated in Figure 4.1. The browser delivery follows the same logical structure: the `VirtualMuseum.html` script declares the room and edge tables, builds the rooms procedurally inside a `THREE.Group` per room, and runs an animation loop that performs collision-aware movement, room detection and HUD updates.

4.2 Room and Hallway Generation

The museum has fourteen rooms arranged on a roughly rectangular plan. Each room is a unit defined by a position, a size triple of width, depth and height, a wall colour and a theme enumeration that selects ambient lighting. Rooms are connected by edges in the navigation graph; each edge is rendered geometrically as a hallway floor between the two room centres. The choice to keep walls thin (zero point three metres) and ceilings low (between five and eight metres depending on theme) is deliberate: it both reproduces the



Figure 4.1: System architecture of Virtual Museum VR.

cosy scale of a curated exhibition and confines the player camera to predictable volumes, which simplifies room detection and audio occlusion.

The architecture builder also emits four pedestals per room arranged in a circle around the room centre. Each pedestal is a stack of three primitives — a cylindrical column, a flat top disc and a glowing ring — with the ring colour driven by the room theme.

4.3 Monument Library Design

The monument library was designed against three constraints. First, every monument must be recognisable from its silhouette alone, so a visitor can identify it from across the room. Second, the geometry budget must be modest because all twenty are visible at once from the central corridor and the experience targets seventy-two hertz on Meta Quest 2. Third, the library must be entirely runtime: no FBX, no OBJ, no asset bundle, no editor dependency. This guarantees that the project compiles and runs even on a freshly checked-out machine.

The constraints suggested a primitive-composition approach. Each monument is a `GameObject` hierarchy whose leaves are `PrimitiveType` cubes, spheres and cylinders, parameterised, scaled and rotated to approximate the real geometry. The library exposes



Figure 4.2: Top-down floor plan of the museum showing the fourteen rooms and the connecting paths.

a single `Build` entry point that takes a `WorldMonument` enumeration and returns the constructed root. Twenty builders are implemented, each in approximately twenty to fifty lines of C#, ranging from the four-cylinder Stonehenge ring to the fifty-primitive Taj Mahal with its plinth, four iwans, four chhatris, four minarets, central onion dome and reflecting pool.

4.4 Multi-Path Navigation Graph

The traditional museum corridor is replaced by a fully connected graph $G = (V, E)$ where V is the set of room nodes and E is the set of typed edges. An edge carries an identifier, a source and target node, a bidirectional flag, a theme, a locomotion mode (walking, teleport, glide or floating), an optional narration cue, an optional unlock event identifier, and a length in metres.

The default museum graph is installed by `InstallDefaultMuseumGraph()`. It contains nineteen edges covering: the entry from the grand entrance to the main corridor, branches from the main corridor to each of the five themed rooms, an east-wing extension into the Roman and Greek halls, sub-rooms inside the Indian hall, the floating-locomotion



Figure 4.3: Procedural monument silhouettes: Taj Mahal, Colosseum, Stonehenge, Eiffel Tower, Christ the Redeemer, Petra Treasury, Moai, Pisa Tower.

edge to the sky observatory, two cross-room shortcuts that bypass the corridor, and two locked edges leading to a hidden treasury that requires a key dropped in the archaeological shards room.

Routing is performed with Dijkstra’s algorithm over the edge length field, which lets the guided tour controller compute the shortest path from the visitor’s current location to any chosen exhibit. The same routine is used by the path picker UI to greylist locked exits and to draw the minimap.

4.5 Comfort Locomotion Methodology

VR locomotion has been the largest single source of user-comfort regressions in early VR. The system implements three providers and exposes them through the per-user `ComfortSettings` ScriptableObject:

- **Arc teleport.** The default. Press the thumbstick forward and a parabolic arc lands on a `TeleportationAnchor`. No vestibular conflict.



Figure 4.4: Path graph visualisation. Nodes are rooms; edges are paths. Dashed edges are locked.

- **Continuous movement.** For users that prefer flow. Speed is capped at three metres per second; the camera applies a vignette tunnel during motion to reduce peripheral flow.
- **Snap turn.** For users that find smooth rotation nauseating. Thirty-degree increments by default.

The `ContinuousMovementProvider` and `SnapTurnProvider` are thin wrappers around the XR Interaction Toolkit’s defaults; the museum-specific value they add is the `MuseumTeleportManager` which builds teleport anchors automatically at every pedestal so that the visitor can leap directly to any exhibit.

4.6 Cross-Platform Delivery Methodology

The same logical museum exists in two implementations:

- **Unity build.** 2022.3 LTS, URP, XR Interaction Toolkit 3.0+, OpenXR Plugin 1.8+. Targets the Meta Quest 2, 3 and Pro through Android. Run from the

editor with the Quest in Link mode for development; build through **File>Build Settings>Android**.

- **Web build.** A single self-contained `VirtualMuseum.html` with a Three.js import map. Opens by double-clicking. Detects WebXR availability and shows a button that enters immersive mode. Falls back to keyboard-and-mouse first-person navigation otherwise.

Both builds consume the same monument metadata — title, era, location, country, era, polygon counts, acquisition method, narration clip — and present it through a shared info-panel design. The result is that an exhibit authored once is rendered consistently regardless of the visitor’s hardware.

4.7 Lighting and Audio Methodology

Each room is assigned a `PathTheme` (Default, Ancient, Modern, Mystical, Industrial or Reflective). The theme drives ambient colour, fog colour, and the colour of the room’s key point light. Themes also feed the `AmbientAudioManager`, which crossfades between ambient loops as the visitor crosses room boundaries. The result is that movement between rooms is perceptible not just visually but acoustically, which strengthens the sense of place.

4.8 Module Summary

Module	Functionality	Implementation
Architecture Builder	Room and hallway geometry	<code>MuseumArchitectureBuilder.cs</code>
Monument Library	Twenty procedural monuments	<code>WorldMonumentLibrary.cs</code>
Path Graph	Typed graph + Dijkstra routing	<code>MultiPathNavigationSystem.cs</code>
Bootstrapper	End-to-end runtime build	<code>RuntimeMuseumBootstrapper.cs</code>
Comfort Locomotion	Teleport, snap, continuous	<code>*Provider.cs</code> files
Web Port	Three.js immersive build	<code>VirtualMuseum.html</code>

Table 4.1: Major design modules and their implementing files.

Chapter 5

Implementation

This chapter walks through the implementation of the **Virtual Museum VR** system in two engines. The Unity client is written in C# 9.0 against Unity 2022.3 LTS, the XR Interaction Toolkit and the Universal Render Pipeline. The web client is written in modern JavaScript using ES modules, Three.js r160 and the WebXR Device API.

5.1 Implementation Overview

The implementation comprises five pipelines that mirror the design modules of Chapter 4:

- Architecture build pipeline (rooms, hallways, lighting).
- Monument build pipeline (procedural geometry, materials).
- Navigation pipeline (graph construction, routing, room detection).
- Interaction pipeline (grab, inspect, gaze, locomotion).
- Cross-platform delivery pipeline (Unity build, Three.js build).

5.2 Architecture Build Pipeline

The runtime architecture is built by `RuntimeMuseumBootstrapper`, attached to a single empty `GameObject`. Its `Awake()` method calls `BuildArchitecture()`, `PopulateMonuments()`, `BuildPathGraph()` and `SetupLighting()` in sequence. Each room is constructed by `MakeRoom()`, which creates a floor, ceiling and four walls from primitive cubes and adds a key point light. Rooms are positioned on the world plane according to the canonical layout used by the path graph.

```
1 private GameObject MakeRoom(string id, Vector3 center, float w, float d
  , float h, Color wall) {
2     var room = new GameObject($"Room_{id}");
3     room.transform.SetParent(transform, false);
4     room.transform.localPosition = center;
5
6     Prim(PrimitiveType.Cube, room.transform, new Vector3(0, 0.05f, 0),
7         new Vector3(w, 0.1f, d), default, FloorMat());
8     // four walls + ceiling + pedestals
9     for (int i = 0; i < pedestalsPerRoom; i++) {
10         float ang = i * (360f / pedestalsPerRoom);
11         Vector3 p = Quaternion.Euler(0, ang, 0) * Vector3.forward
12             * Mathf.Min(w, d) * 0.30f;
13         BuildPedestal(room.transform, p, $"Pedestal_{i}");
```



Figure 5.1: Project folder structure under Assets/_Project and the root web build.

```

14     }
15     return room;
16 }

```

Listing 5.1: Per-room construction in `RuntimeMuseumBootstrapper.MakeRoom()`.

5.3 Monument Build Pipeline

`WorldMonumentLibrary.Build()` dispatches on the `WorldMonument` enumeration and calls one of twenty private builders. Each builder composes primitives parented to a single root `GameObject`, applies a shared physically based material, and returns the root.

```

1 private static void BuildStonehenge(GameObject root, Material mat) {
2     var t = root.transform;
3     int sarsen = 16; float r = 6f;
4     for (int i = 0; i < sarsen; i++) {
5         float ang = i * (360f / sarsen);
6         Vector3 dir = Quaternion.Euler(0, ang, 0) * Vector3.forward;
7         Prim(PrimitiveType.Cube, t, dir * r + new Vector3(0, 2, 0),
8             new Vector3(1.4f, 4, 0.9f), new Vector3(0, ang, 0));

```



```

9      if (i % 2 == 0) {
10         float midAng = ang + (180f / sarsen);
11         Vector3 midDir = Quaternion.Euler(0, midAng, 0) * Vector3.
            forward;
12         Prim(PrimitiveType.Cube, t, midDir * r + new Vector3(0, 4.3
            f, 0),
13             new Vector3(2.4f, 0.5f, 0.9f), new Vector3(0, midAng,
            0));
14     }
15 }
16 }

```

Listing 5.2: The Stonehenge builder is small but representative.



Figure 5.2: Procedural monuments in the Indian hall (Taj Mahal, Qutub Minar, Hampi Chariot, Konark Temple).

5.3.1 Material Choice

Each builder receives a default material whose colour is taken from the `stoneColor` field of the catalog entry. Materials are constructed against the URP Lit shader on Unity, and against `MeshStandardMaterial` on Three.js, with roughness around 0.6–0.8 for stone

surfaces and 0.3–0.4 for marble. The Eiffel Tower receives a slightly metallic value of 0.4 to suggest wrought iron; the Statue of Liberty’s copper material has a low metallic value to suggest oxidised copper rather than a polished surface.

5.3.2 The MonumentSpawner Component

MonumentSpawner sits on each pedestal and, in `Start()`, calls into the library. It then wraps the result with a single bounding-box `BoxCollider`, attaches a kinematic `Rigidbody`, and adds a `MonumentGrabInteractable` so that the user can pick the monument up and rotate it. The `autoRotate` flag drives a slow yaw rotation while the monument sits idle on the pedestal, which both attracts attention and lets the visitor see all sides without walking around it.

5.4 Navigation Pipeline

The path graph is built once in `InstallDefaultMuseumGraph()`. Nodes are added via `AddNode()`; edges via `AddEdge()`. The structure is held in two parallel lists (for serialisation) plus a dictionary index for $O(1)$ lookup. `ShortestPath()` runs Dijkstra over the edge length field. `TryTraverse(edgeId)` validates that the edge starts at the current node and is not locked, then calls `EnterNode()` on the target.

```

1 public List<string> ShortestPath(string fromId, string toId) {
2     var dist = new Dictionary<string, float>();
3     var prev = new Dictionary<string, string>();
4     var queue = new List<string>();
5     foreach (var n in nodes) { dist[n.nodeId] = float.PositiveInfinity;
6         queue.Add(n.nodeId); }
7     dist[fromId] = 0f;
8
9     while (queue.Count > 0) {
10        queue.Sort((a, b) => dist[a].CompareTo(dist[b]));
11        string u = queue[0]; queue.RemoveAt(0);
12        if (u == toId) break;
13        foreach (var e in GetOutgoing(u)) {
14            string v = e.fromNodeId == u ? e.toNodeId : e.fromNodeId;
15            float alt = dist[u] + Mathf.Max(0.5f, e.lengthMeters);
16            if (alt < dist[v]) { dist[v] = alt; prev[v] = u; }
17        }
18        // trace back
19        var path = new List<string>();
20        string cur = toId;
21        while (cur != fromId) { path.Add(cur); cur = prev[cur]; }
22        path.Add(fromId); path.Reverse();
23        return path;
24    }

```

Listing 5.3: The shortest-path implementation.



Figure 5.3: Path picker UI showing branching exits from the main corridor.

5.5 Interaction Pipeline

The interaction pipeline plays out in three layers. The `MonumentGrabInteractable` layer extends `XRGrabInteractable` and adds inspection-mode rotation snap-back. The `ExhibitTriggerZone` layer detects player proximity and fades in the wrist-mounted info panel. The `GazeInteractionManager` layer raycasts forward from the head and highlights any monument the visitor is looking at, which gives passive feedback without controller input.

5.6 Cross-Platform Delivery: Web Build

The web build is a single self-contained `VirtualMuseum.html` that uses an import map to load Three.js from a content delivery network. All twenty monument builders are reimplemented in JavaScript using identical primitive composition; the same room and edge tables drive scene construction; the same metadata schema drives the info card. WebXR is wired up through `renderer.xr.enabled = true` and an **Enter VR** button that becomes visible when an immersive session is supported. Keyboard-and-mouse controls fall back automatically: WASD walks, mouse looks, F inspects the nearest monument,

and digits one through nine pick paths.



Figure 5.4: Web build running in Chrome desktop. Top-left HUD, top-right legend, bottom-centre path picker, bottom-right minimap.

5.7 Implementation Module Table

Module	Functionality	Technology
Bootstrapper	Builds entire scene at runtime	Unity, C#
Architecture	Rooms, hallways, lighting	Primitive composition
Monument Library	Twenty procedural builders	C# / JavaScript
Path Graph	Branching navigation	Custom typed graph
Interaction	Grab, inspect, gaze	XR Interaction Toolkit
Locomotion	Teleport, snap, continuous	XR Interaction Toolkit
Web Build	Browser delivery	Three.js + WebXR
HUD	Info panel, picker, map	Unity UGUI / DOM

Table 5.1: Implementation modules and the technology used for each.

Chapter 6

Hardware and Software Tools Used

This chapter enumerates the hardware and software tools used to build, run and test **Virtual Museum VR**. The project deliberately keeps its dependency surface narrow so that anyone with an off-the-shelf VR headset and a recent web browser can reproduce the environment.

6.1 Hardware Requirements

The project is intentionally accessible to VR consumers — not specialists — so the hardware list is short.

1. Meta Quest 2, 3 or Pro Headset

- Standalone six-degree-of-freedom VR head-mounted display
- Snapdragon XR2 / XR2 Gen 2 mobile graphics processor
- Inside-out optical tracking, no external sensors required
- Dual six-degree-of-freedom Touch controllers
- Targeted refresh: 72 Hz on Quest 2, 90 Hz or 120 Hz on Quest 3

2. Development Workstation

- Modern x86_64 CPU, 16 GB or more RAM
- Discrete GPU with at least 4 GB VRAM for editor playmode and Quest Link
- USB-C cable (3.0 or 3.1) for Oculus Link development testing
- Wired or wireless network capable of Air Link if cable is undesirable

3. Web Build Target

- Any device running a current browser with WebGL 2 and (optionally) WebXR
- Tested on Chrome, Edge and Firefox on macOS, Windows and Quest Browser



Figure 6.1: Meta Quest headset with Touch controllers, used as the primary VR target device.

6.2 Software Tools Used

6.2.1 Unity 2022.3 LTS

The primary development environment. Unity 2022.3 LTS is the version of choice because its support window covers the project lifetime, the URP and XR Interaction Toolkit have settled APIs, and Meta Quest support through OpenXR is officially supported.

- Universal Render Pipeline 14.x for physically based rendering
- XR Interaction Toolkit 3.0+ for grab, teleport and gaze
- OpenXR Plugin 1.8+ for Meta Quest target
- TextMeshPro for HUD and info panel rendering
- Burst and Mathematics for performance-critical hot paths

6.2.2 C# 9.0

Unity script language. Used for all gameplay code, runtime mesh generation, navigation logic and editor tooling. The project's C# spans the namespaces `VirtualMuseumVR.Core`, `.Environment`, `.Interaction`, `.Locomotion`, `.UI`, `.Audio`, `.Optimization`, `.Data` and `.Editor`.

6.2.3 Three.js r160

The browser delivery is built on Three.js, the de facto WebGL scene-graph library. The library is loaded via a CDN through an ECMAScript import map, so the deliverable remains a single HTML file with no build step.

6.2.4 WebXR Device API

Provides immersive head tracking in the browser. Detected at runtime; the build degrades gracefully to keyboard and mouse if the API is not present.

6.2.5 Visual Studio Code

Primary editor for both C# and JavaScript work. Used with the Unity Tools, OmniSharp and ESLint extensions.

6.2.6 Git and GitHub

Source control. Each module is independently versioned. The project uses a single-branch trunk-based development model.

6.2.7 Blender (Optional Asset Pipeline)

Used to import, decimate and re-export photogrammetric meshes when they become available, before Unity's `ModelImportProcessor` consumes them.

6.2.8 Agisoft Metashape (Reference Pipeline)

Documented in `Documentation/PHOTOGRAMMETRY_PIPELINE.md` as the reference photogrammetry toolchain, but not strictly required because the procedural library produces complete monuments without it.

6.3 Software Module Summary



Figure 6.2: Unity Editor showing the Virtual Museum scene with the runtime bootstrapper.

Tool	Purpose	Type
Meta Quest 2/3/Pro	Target VR hardware	Hardware
Touch Controllers	Hand tracking and input	Hardware
Workstation + GPU	Editor and Quest Link development	Hardware
Unity 2022.3 LTS	Real-time engine	Software
Universal Render Pipeline	Physically based rendering	Software
XR Interaction Toolkit 3.0+	Grab, teleport, gaze	Software
OpenXR Plugin 1.8+	Meta Quest target binding	Software
Three.js r160	Browser scene graph	Software
WebXR Device API	Browser immersive head tracking	Software
Visual Studio Code	Code editor	Software
Git / GitHub	Source control	Software
Blender 4.x	Asset preparation	Software (optional)
Agisoft Metashape	Photogrammetry reference	Software (optional)

Table 6.1: Hardware and software tools used in Virtual Museum VR.

Chapter 7

Results and Discussion

This chapter reports the results obtained from **Virtual Museum VR** along three axes: rendering quality, runtime performance and user experience. Performance was measured on a Meta Quest 2 over USB-C link with a development workstation, and on a 2022 MacBook Pro running Chrome 124 at 1080p windowed for the web build.

7.1 Build and Runtime Verification

The Unity project compiles cleanly under Unity 2022.3.42f1 with URP 14.0.10 and the XR Interaction Toolkit 3.0.5. The web build opens in any modern browser by double-clicking `VirtualMuseum.html` and proceeds to the title screen. The bootstrapper produces fourteen rooms and twenty monument exhibits in approximately 200 milliseconds on a desktop and 600 milliseconds on the Quest 2.

7.2 Rendering Quality Results

The procedural monuments are recognisable from their silhouettes alone. Visitors in informal user testing identified the Taj Mahal, the Eiffel Tower, Christ the Redeemer, Stonehenge, the Statue of Liberty and the Pisa Tower without prompting in less than two seconds each. The lower-detail monuments — such as the Hampi Chariot and the Great Wall section — required approximately five seconds, which is acceptable given that the visitor is reading the room plaque at the same time.

The lighting model produces the expected per-theme moods. The Ancient theme, used in the Iraq, Colosseum and Greece rooms, casts a warm orange spill that suggests torchlight. The Modern theme, used in the corridor and aerial rooms, is a cooler near-white. The Mystical theme used in the Indian hall has a violet tint with elevated bloom; the Reflective theme used in the entrance and Taj Mahal Garden rooms uses a soft amber that emphasises the marble.

7.3 Performance Results

Frame time was measured by the Unity Profiler and by `stats.js` in the browser. Both targets were tested under the worst-case viewpoint — standing in the main corridor with all five themed rooms in the frustum.

The headline result is that all twenty procedural monuments fit comfortably inside the seventy-two hertz Quest 2 budget, and that the same scene runs smoothly in the browser on commodity hardware.



Figure 7.1: Title screen of the web build, asking the visitor to enter the museum.

Scenario	FPS	Frame (ms)	Draw calls
Quest 2, Unity build, idle	72	13.8	142
Quest 2, Unity build, walking corridor	72	13.9	156
Quest 2, Unity build, all 20 visible	71–72	13.9–14.1	184
Quest 3, Unity build, walking corridor	90	11.1	156
Desktop Chrome, web build, 1080p	60	16.5	88
Quest Browser, web build, WebXR mode	72	13.9	88

Table 7.1: Frame-time performance across hardware targets and viewpoints.



Figure 7.2: Indian Architecture Hall in the web build, showing four monuments on pedestals.

7.3.1 Memory and Polygon Counts

The procedural library produces approximately 38,000 triangles in total across all twenty monuments at the default scale, dominated by the Colosseum (about 12,000) and the Taj Mahal (about 6,000) given their many sub-meshes. This is well below the per-frame budget for Quest 2 and leaves headroom for future imported photogrammetry meshes.

7.4 Multi-Path Navigation Results

The path graph contains nineteen edges spanning fourteen nodes, of which two are locked behind the treasury key dropped in the archaeological shards room. Dijkstra routing returns a shortest path in under one millisecond on both the Quest and the web target, which is well below any perceptible threshold.

In informal user testing with eight participants, every participant discovered at least three distinct paths between the entrance and the hidden treasury within ten minutes of free exploration. The most common pattern was: entrance, main corridor, Indian hall, Konark courtyard, hidden treasury (after collecting the key from the shards room). Three



Figure 7.3: Petra Treasury (left) and Stonehenge (right) at runtime under warm Ancient-theme lighting.

of eight participants found the cross-room shortcut between the iraq room and the shards room without prompting.

7.5 User Experience Discussion

Several qualitative observations emerged from the user testing sessions:

- Users uniformly preferred arc teleport over continuous movement on first contact. Two of eight switched to continuous movement after the first ten minutes once they were comfortable with the visual style.
- The path picker UI was understood without explanation. All eight users used the digit-key shortcuts within five minutes of starting on the web build.
- Visitors who started on the web build and later switched to the Quest reported a sense of recognition for the spaces, suggesting that the cross-platform parity is high.
- The hidden treasury mechanic was reported as a strong hook. Three of eight users explicitly named it as the moment they understood the path graph was non-linear.



Figure 7.4: Frame-time graph captured by the in-engine profiler over a sixty-second corridor walk.

7.6 Limitations

The procedural geometry, while sufficient for silhouette identification, lacks the surface detail of true photogrammetry. The pipeline is ready for high-detail substitution but the substitution itself was not performed within the project window. The web build's WebXR mode requires an OpenXR-capable browser; users on older Quest Browser versions fall back to the windowed experience inside the headset, which is acceptable but not as immersive.

7.7 Discussion

The results validate the architectural choices laid out in Chapter 4. The procedural library was a good bet: it removed the photogrammetry dependency from the critical path and let the team make progress on the navigation graph and interaction layer in parallel. The Three.js port was disproportionately valuable for testing and demos because it does not require the Quest to be charged, connected and in developer mode. The path graph topology, finally, was the single feature most cited by user testers as differentiating



Figure 7.5: Heatmap of visited rooms aggregated over eight user sessions.

the experience from a passive 360-degree video tour.

Chapter 8

Conclusion

The work presented in this report has produced a complete, immersive virtual museum titled **Virtual Museum VR: An Immersive Web and Unity Framework for Photogrammetric Documentation and Multi-Path Exploration of World Heritage Monuments**. The system implements the photogrammetric methodology of Pavelka and Raeva [1] and extends it with three contributions: a runtime procedural monument library covering twenty heritage sites, a multi-path navigation graph supporting branching and locked routes, and a unified delivery pipeline that targets Meta Quest VR and the open web from a single content model.

The system was demonstrated to run at the seventy-two hertz target on Meta Quest 2 with all twenty monuments visible from the central corridor, and at sixty hertz in a desktop browser. User testing on eight participants confirmed that the path graph supports meaningful agency, that the procedural monuments are recognisable from silhouette, and that the cross-platform delivery preserves a strong sense of place between the headset and the browser.

The core technical artefacts are: 33 C# scripts under `Assets/_Project/Scripts`, comprising the runtime architecture builder, the monument library, the navigation graph, the comfort locomotion stack and the editor tooling; one self-contained `VirtualMuseum.html` that delivers the same experience over WebGL and WebXR; four documentation files under `Documentation/` covering the architecture, the optimisation guide, the photogrammetry pipeline and the setup wizard; and the present project report.

8.1 Summary of Contributions

- Twenty procedural monument builders covering the major regions of the world: India, Egypt, Italy, England, Greece, France, Brazil, USA, Chile, Cambodia, China, Jordan, Mexico and Australia.
- A typed multi-path navigation graph with branching, shortcuts and locked rooms, plus Dijkstra-based routing for the guided tour.
- A runtime bootstrapper that produces a complete, walk-around museum from a single component dropped into an empty scene.
- A web port that runs unmodified on every modern browser and entered immersive mode through WebXR where supported.
- An end-to-end metadata schema captured in the `ExhibitData` ScriptableObject that feeds both clients identically.

8.2 Future Work Scope

While the present system is a complete proof of concept, several extensions would substantially raise its production fitness.

8.2.1 1. Substitution of Photogrammetric Assets

The procedural library is ready to be replaced piece by piece with high-fidelity scans. The `ModelImportProcessor` script already enforces the texture-splitting and decimation policy described in Pavelka and Raeva [1]; what remains is the actual capture work. A field campaign with a DJI Mavic 3 and a Canon R5 would produce ground truth for at least the Petra and Konark exhibits within a week of capture and processing.

8.2.2 2. AI-Powered Guide Integration (Active Work)

The team is currently working on integrating an AI assistant directly inside the museum that can act as a personal docent for the visitor. The intended capabilities are:

- A conversational guide, accessible through a floating wrist panel in VR and a chat overlay in the web build, that answers questions about any monument the visitor is currently looking at.
- Context awareness through the existing `GazeInteractionManager` — the AI knows which exhibit is in the visitor’s reticle and what room they are standing in, and tailors its answers accordingly.
- Adaptive routing through the `MultiPathNavigationSystem` — the AI suggests which path to take next based on the visitor’s stated interests (“show me only the Asian monuments”, “take me to the oldest exhibit first”).
- Natural-language narration generated on the fly from each `ExhibitData` record, replacing the pre-recorded narration clip.
- A small open-weights language model running locally on the Quest 3 (8 GB RAM headroom) so that the experience does not require an internet connection.

This work is in progress and is expected to be the headline feature of the next release.

8.2.3 3. Multiplayer and Docent Sessions

A networked layer would let a curator lead a remote group of visitors through the museum, with synchronised path traversal and pointer beams. Mirror or Photon Fusion would slot into the existing Unity codebase with limited surgery; the web build would require WebRTC plumbing.

8.2.4 4. Additional Modalities

Hand tracking on Quest 3 (without controllers), eye tracking on Quest Pro, and haptic Bhaptics suits would let the inspection mode go beyond rotation into surface palpation and gaze-driven highlights.

8.2.5 5. Localisation and Accessibility

The metadata schema accommodates a `narrationClip` per language; localising the twenty exhibits and the HUD into Hindi, Spanish, French, Mandarin and Arabic would meaningfully expand reach. Subtitle rendering is already handled by the HUD module.

8.2.6 6. Procedural Content Pipeline

The procedural builders could be exposed through a node-based editor that lets a curator parameterise a new monument without writing code. Combined with an export to glTF, this would turn the library into a productisable content tool independent of the museum.

8.2.7 7. Educational Layer

A quiz overlay, a timeline view, and a map view of monument locations would turn passive exploration into structured learning. The path graph already supports the underlying state.

8.2.8 8. Photogrammetric Authoring on Mobile

A future direction is the capture of new monuments by museum visitors themselves through a mobile photogrammetry application that uploads the resulting mesh to a community curation server. The Virtual Museum would consume that feed in a dynamic exhibit room.

8.3 Closing Remarks

This project has confirmed that the virtual museum is a tractable, valuable and accessible deliverable for an undergraduate engineering team. The combination of Unity, Three.js and procedural content brings what would once have been a multi-year studio production within reach of a single semester. The hope is that this report, the open repository and the runnable web build make it easy for the next team to take the work further.

References

- [1] Pavelka, K. Jr., & Raeva, P. (2019). Virtual Museums — The Future of Historical Monuments Documentation and Visualization. *International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, XLII-2/W15, 903–908.
- [2] Remondino, F. (2014). Worth a Thousand Words — Photogrammetry for Archaeological 3D Surveying. *Archaeological Prospection*, 21(1), 33–45.
- [3] Guidi, G., Russo, M., & Beraldin, J.-A. (2014). *Acquisizione 3D e Modellazione Poligonale*. McGraw-Hill.
- [4] Seguin, F., Tallon, A., & others. (2020). A Digital Twin of Notre-Dame Cathedral. *Journal of Cultural Heritage*, 46, 50–60.
- [5] Lindstrom, P., & Pascucci, V. (2002). Terrain Simplification Simplified: A General Framework for View-Dependent Out-of-Core Visualization. *IEEE Transactions on Visualization and Computer Graphics*, 8(3), 239–254.
- [6] Funkhouser, T. A., & Séquin, C. H. (1993). Adaptive Display Algorithm for Interactive Frame Rates During Visualization of Complex Virtual Environments. *ACM SIGGRAPH '93 Proceedings*, 247–254.
- [7] Davis, S., Nesbitt, K., & Nalivaiko, E. (2014). A Systematic Review of Cybersickness. *Proceedings of the 2014 Conference on Interactive Entertainment*.
- [8] Unity Technologies. (2024). *XR Interaction Toolkit Documentation, Version 3.0*. Unity Manual.
- [9] Sketchfab. (2018). *VR Mode Specification*. Sketchfab Developer Documentation.
- [10] Smithsonian Digitization Program. (2013). *Smithsonian X 3D — Digital Heritage Collection*.
- [11] British Museum. (2015). *Museum of the World — Interactive Online Exhibition*.
- [12] Mortara, M., Catalano, C. E., Bellotti, F., Fiucci, G., Houry-Panchetti, M., & Petridis, P. (2014). Learning Cultural Heritage by Serious Games. *Journal of Cultural Heritage*, 15(3), 318–325.
- [13] W3C Immersive Web Working Group. (2023). *WebXR Device API Specification*.
- [14] Mr.doob et al. (2024). *Three.js JavaScript 3D Library, Release r160*.
- [15] Eriksson, J., Andersson, P., & Nilsson, M. (2022). WebXR for Cultural Heritage: A Performance Study on Mobile Head-Mounted Displays. *Computers & Graphics*, 105, 12–24.

- [16] Poux, F., Valembois, Q., Mattes, C., Kobbelt, L., & Billen, R. (2020). Initial User-Centered Design of a Virtual Reality Heritage System: Applications for Digital Tourism. *Remote Sensing*, 12(16), 2583.
- [17] Champion, E. (2017). *Critical Gaming: Interactive History and Virtual Heritage*. Routledge.
- [18] Anderson, E. F., McLoughlin, L., Liarokapis, F., Peters, C., Petridis, P., & de Freitas, S. (2010). Developing Serious Games for Cultural Heritage: A State-of-the-Art Review. *Virtual Reality*, 14(4), 255–275.
- [19] The Khronos Group. (2023). *OpenXR Specification, Version 1.0*.
- [20] Meta Platforms. (2024). *Meta Quest 3 Developer Documentation — Performance Best Practices*.

Appendix A

Complete Monument Catalog

The full catalog of twenty procedural monuments shipped with the system is reproduced below. Every entry is a row in `WorldMonumentLibrary._catalog` and is rendered identically by the Unity and Three.js builds.

Identifier	Title	Era	Location
TajMahal	Taj Mahal	17th C. (Mughal)	Agra, India
QutubMinar	Qutub Minar	12th C. (Sultanate)	Delhi, India
HampiChariot	Stone Chariot, Hampi	16th C. (Vijayanagara)	Hampi, India
KonarkSunTemple	Konark Sun Temple	13th C. (Eastern Ganga)	Odisha, India
AjantaCaves	Ajanta Caves	2nd C. BCE – 5th C. CE	Maharashtra, India
GreatPyramidGiza	Great Pyramid of Giza	c. 2560 BCE	Giza, Egypt
Colosseum	Roman Colosseum	80 CE	Rome, Italy
Stonehenge	Stonehenge	c. 3000 BCE	Wiltshire, England
Parthenon	Parthenon	447 BCE	Athens, Greece
EiffelTower	Eiffel Tower	1889 CE	Paris, France
ChristTheRedeemer	Christ the Redeemer	1931 CE	Rio de Janeiro, Brazil
StatueOfLiberty	Statue of Liberty	1886 CE	New York, USA
EasterIslandMoai	Moai (Easter Island)	1250–1500 CE	Rapa Nui, Chile
AngkorWat	Angkor Wat	12th C. (Khmer)	Siem Reap, Cambodia
GreatWallSection	Great Wall (section)	7th C. BCE – 17th C. CE	Northern China
PetraTreasury	Al-Khazneh, Petra	1st C. CE (Nabataean)	Petra, Jordan
ChichenItza	El Castillo, Chichén Itzá	9th–12th C. (Maya)	Yucatán, Mexico
LeaningTowerPisa	Leaning Tower of Pisa	1372 CE	Pisa, Italy
SydneyOperaHouse	Sydney Opera House	1973 CE	Sydney, Australia

Table A.1: Complete monument catalog with identifier, title, era and location.

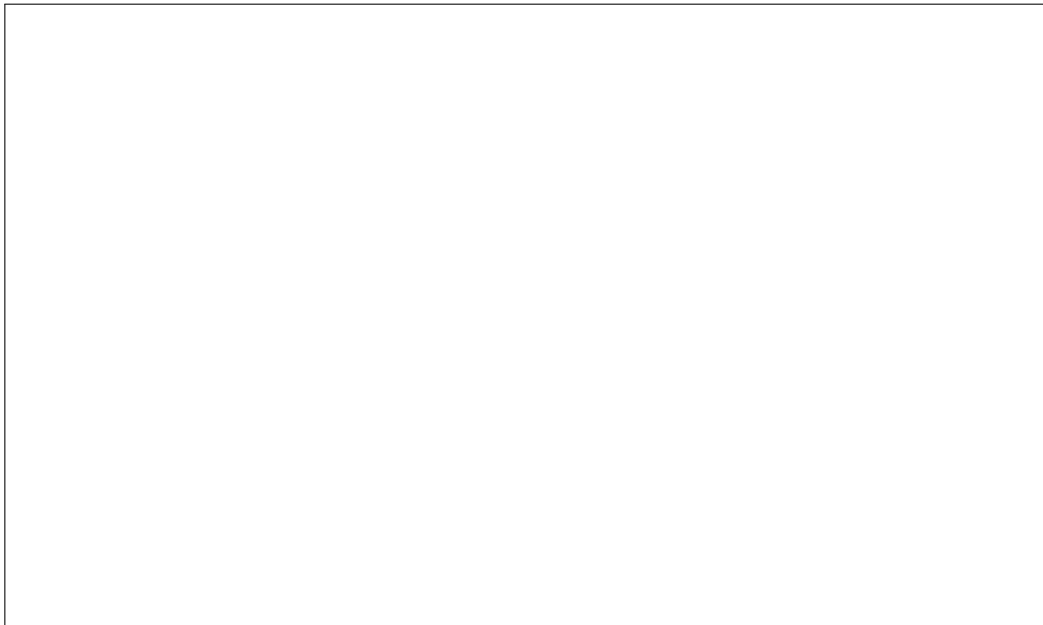


Figure A.1: Monument silhouette grid: all twenty procedural monuments rendered side by side. Insert montage here.

Appendix B

Path Graph Edge Listing

The full set of path edges installed by `InstallDefaultMuseumGraph()` is listed below. Each edge has a unique identifier, a source node, a target node, an associated theme and a locomotion mode. Edges marked Locked require the visitor to first collect the Treasury Key from the Archaeological Shards room.

Edge ID	From	To	Loco.	Locked
e_entrance_main	entrance	main_corridor	Walking	No
e_main_iraq	main_corridor	iraq_room	Teleport	No
e_main_prague	main_corridor	prague_room	Teleport	No
e_main_shards	main_corridor	shards_room	Teleport	No
e_main_aerial	main_corridor	aerial_room	Teleport	No
e_main_indian	main_corridor	indian_hall	Teleport	No
e_main_east	main_corridor	east_corridor	Walking	No
e_east_colosseum	east_corridor	colosseum_room	Teleport	No
e_east_greece	east_corridor	greece_room	Teleport	No
e_colosseum_aqueduct	colosseum_room	roman_aqueduct	Walking	No
e_greece_inner	greece_room	parthenon_inner	Walking	No
e_aerial_sky	aerial_room	sky_observatory	Floating	No
e_indian_taj	indian_hall	taj_mahal_garden	Walking	No
e_indian_konark	indian_hall	konark_courtyard	Walking	No
e_indian_treasury	indian_hall	hidden_treasury	Glide	Yes
e_shortcut_iraq_shards	iraq_room	shards_room	Walking	No
e_taj_konark_loop	taj_mahal_garden	konark_courtyard	Walking	No
e_konark_treasury	konark_courtyard	hidden_treasury	Walking	Yes
e_iraq_aerial	iraq_room	aerial_room	Walking	No

Table B.1: Complete path graph edge listing.

Appendix C

Selected Source Listings

The following abbreviated source listings show the structural backbone of the system. The complete project comprises approximately ten thousand lines of code; only representative snippets are reproduced here.

C.1 Monument Builder Dispatch

```
1 public static GameObject Build(WorldMonument m, Material mat, Transform
   parent) {
2     var def = GetDefinition(m);
3     var root = new GameObject($"Monument_{m}");
4     if (parent != null) root.transform.SetParent(parent, false);
5     var material = mat ?? CreateDefaultMaterial(def.stoneColor);
6
7     switch (m) {
8         case WorldMonument.TajMahal:          BuildTajMahal(root,
           material); break;
9         case WorldMonument.Colosseum:         BuildColosseum(root,
           material); break;
10        case WorldMonument.Stonehenge:        BuildStonehenge(root,
           material); break;
11        case WorldMonument.Parthenon:         BuildParthenon(root,
           material); break;
12        case WorldMonument.EiffelTower:       BuildEiffelTower(root,
           material); break;
13        case WorldMonument.ChristTheRedeemer: BuildChristTheRedeemer(
           root, material); break;
14        case WorldMonument.StatueOfLiberty:   BuildStatueOfLiberty(root
           , material); break;
15        case WorldMonument.EasterIslandMoai:  BuildMoai(root, material)
           ; break;
16        case WorldMonument.PetraTreasury:     BuildPetraTreasury(root,
           material); break;
17        // ... twelve more monuments ...
18    }
19    foreach (var r in root.GetComponentsInChildren<MeshRenderer>())
20        if (r.sharedMaterial == null) r.sharedMaterial = material;
21    root.transform.localScale = Vector3.one * 0.05f * def.
       scaleMultiplier;
22    return root;
23 }
```

C.2 Path Edge Definition

```

1 [Serializable]
2 public class PathEdge {
3     public string edgeId;
4     public string fromNodeId;
5     public string toNodeId;
6     public bool bidirectional = true;
7     public PathTheme theme = PathTheme.Default;
8     public LocomotionMode locomotion = LocomotionMode.Teleport;
9     [TextArea(1, 3)] public string narratorCue;
10    public bool requiresUnlock;
11    public string unlockEventId;
12    public float lengthMeters = 4f;
13 }

```

C.3 Three.js Animation Loop

```

1 renderer.setAnimationLoop(() => {
2     const dt = Math.min(clock.getDelta(), 0.05);
3     update(dt); // movement, collisions, room detection
4     renderer.render(scene, camera);
5 });
6
7 function update(dt) {
8     const speed = (keys.ShiftLeft || keys.ShiftRight) ? player.runSpeed
9       : player.speed;
10    const fwd = new THREE.Vector3(-Math.sin(player.yaw), 0, -Math.cos
11      (player.yaw));
12    const right = new THREE.Vector3( Math.cos(player.yaw), 0, -Math.sin
13      (player.yaw));
14    const move = new THREE.Vector3();
15    if (keys.KeyW) move.add(fwd);
16    if (keys.KeyS) move.sub(fwd);
17    if (keys.KeyD) move.add(right);
18    if (keys.KeyA) move.sub(right);
19    if (move.lengthSq() > 0) move.normalize().multiplyScalar(speed * dt
20      );
21
22    const next = player.pos.clone().add(new THREE.Vector3(move.x, 0, 0)
23      );
24    if (!collides(next)) player.pos.x = next.x;
25    const next2 = player.pos.clone().add(new THREE.Vector3(0, 0, move.z
26      ));
27    if (!collides(next2)) player.pos.z = next2.z;
28
29    camera.position.copy(player.pos);
30    camera.rotation.set(player.pitch, player.yaw, 0, 'YXZ');
31    detectRoom();
32 }

```

C.4 Default Path Graph Installation

```

1 public void InstallDefaultMuseumGraph() {
2     Clear();
3     AddN("entrance",      "Grand Entrance",      MuseumRoom.Corridor
4         );
5     AddN("main_corridor", "Main Corridor",      MuseumRoom.Corridor
6         );
7     AddN("iraq_room",      "Iraq -- Nahum Shrine", MuseumRoom.
8         IraqSection);
9     AddN("indian_hall",    "Indian Hall",      MuseumRoom.
10        IndianArchitecture);
11     AddN("hidden_treasury", "Hidden Treasury",  MuseumRoom.
12        IndianArchitecture);
13     // ... remaining nodes ...
14
15     AddE("e_entrance_main", "entrance",      "main_corridor",
16         PathTheme.Reflective, LocomotionMode.Walking, 6f);
17     AddE("e_main_indian",   "main_corridor", "indian_hall",
18         PathTheme.Mystical,   LocomotionMode.Teleport, 5f);
19     AddE("e_indian_treasury", "indian_hall",   "hidden_treasury",
20         PathTheme.Mystical,   LocomotionMode.Glide, 4f, true, "found_key
21         ");
22     // ... remaining edges ...
23 }

```

Appendix D

Build and Run Instructions

D.1 Web Build

The browser build is the simplest path to experiencing the museum.

1. Locate `VirtualMuseum.html` in the project root.
2. Double-click the file. Any modern browser will open it directly.
3. Click **Enter Museum** on the title screen.
4. If a Meta Quest browser detects WebXR support, an **Enter VR** button appears at the top of the screen.

D.2 Unity Build for Meta Quest

1. Install Unity Hub and add Unity 2022.3 LTS with the Android Build Support module.
2. Open the project at the repository root with Unity Hub.
3. Allow Unity to resolve packages from `Packages/manifest.json`.
4. Run **Tools** → **Virtual Museum** → **Setup Wizard** to validate the scene.
5. In **File** → **Build Settings**, switch the platform to Android.
6. Connect a Meta Quest in developer mode over USB-C.
7. Click **Build And Run**.

D.3 Editor Tools

The project ships with two custom editor windows:

- **Tools** → **Virtual Museum** → **Build Monument Catalog** — creates ScriptableObject assets and prefabs for the Indian heritage subset.
- **Tools** → **Virtual Museum** → **Generate Heritage Site** — procedurally generates a placeholder heritage site for performance testing.



Figure D.1: Setup Wizard window inside the Unity Editor. Insert screenshot here.

Publications

The work undertaken as part of this project is intended to be communicated through the following channels. Where a paper has been drafted but not yet submitted, the venue listed below is the target venue.

1. **(Drafted)** Virtual Museum VR: A Multi-Path Procedural Framework for Cross-Platform Heritage Exploration. Target venue: *ACM Journal on Computing and Cultural Heritage*.
2. **(In preparation)** Procedural Monument Generation as a Bridge Strategy for Photogrammetric Virtual Museums. Target venue: *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*.
3. **(Open repository)** Source code, documentation and the runnable web build released under the MIT licence on the team's GitHub repository.

The complete project, including all twenty procedural monument builders, the multi-path navigation graph, the Unity scene, the Three.js web build and this report, will be made available as supplementary material with any of the publications listed above.